

3.4 GPT and BERT

Same block. Causal next-token versus masked tokens. Contextual, not a static table.

These notes match the lecture slides. Use **(slides)** on the course hub for the deck.

GPT and BERT are the same block with different **masks** and **objectives**. That choice is the difference between a generator and a bidirectional encoder. The fork is the objective: causal language modeling versus masked language modeling.

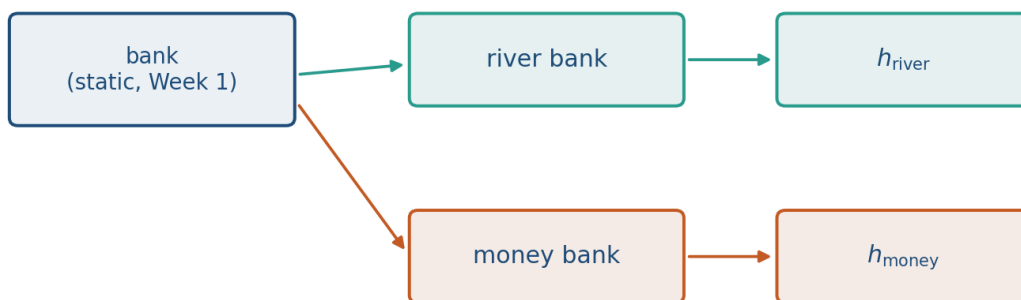
First time this method appears. GPT and BERT are two uses of the same block.

What. GPT: causal next-token (left to right). BERT: masked tokens, bidirectional context. Both produce **contextual** vectors, not a static table (Week 1). **Why.** Language modeling (GPT) is the pretrain that later becomes ChatGPT. BERT-style MLM is a bidirectional encoder for classification and span tasks. **Architecture.** Same transformer block. GPT masks the future. BERT masks random token ids and reconstructs them. Heads differ (LM vs CLS/span). **How.** Train on unlabeled text. After pretrain, GPT generates; BERT is usually fine-tuned with a small head. **Formula.** GPT: $\prod_t p(w_t | w_{<t})$. BERT: reconstruct masked positions, not a full joint factorization of the sentence. **Tradeoffs.** + Contextual geometry. – GPT cannot see the future inside a prompt token; BERT is not a natural generator. “Transformer” is the block; GPT/BERT is the **objective**.

1. Static versus contextual

Week 1’s embedding table gave *bank* **one** vector. A transformer gives *bank* a vector that depends on the sentence: river bank versus money bank. That is a **contextual** representation. ELMo did this with LSTMs; GPT and BERT do it with the Week 3 block.

A transformer embedding is contextual: the same type, two vectors



Pretrain once on unlabeled text, then fine-tune a small head (or the whole stack) on a labeled task.

Pretrain once, fine-tune many times.

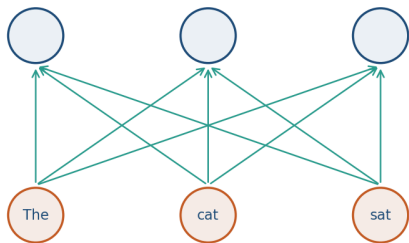
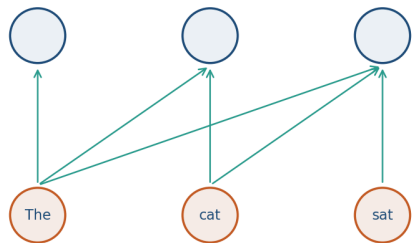
2. Two attention patterns

GPT (decoder-only): each token may attend to itself and the past. The pretraining job is **next-token** (causal language modeling). Sampling is ancestral: emit t , then condition on it.

BERT (encoder-only): every token may attend to every token. The pretraining job is **masked language modeling** (and originally next-sentence prediction; later work often drops NSP). You cannot sample a paragraph left-to-right without extra machinery.

GPT: causal (next token)

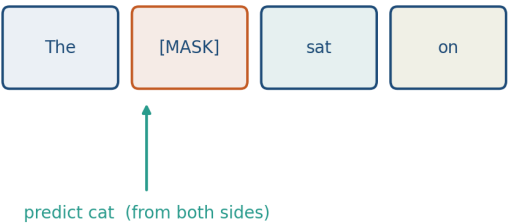
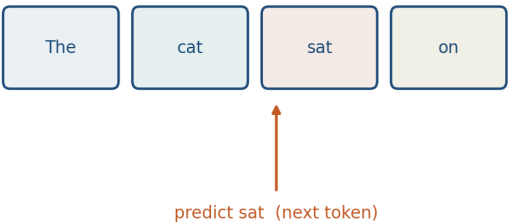
BERT: bidirectional (masked token)



Causal mask: a lower-triangular pattern of legal scores. BERT's “mask” is a **token** [MASK] in the input, not a future-key mask. Those two uses of the word confuse people all semester.

GPT: causal language model

BERT: masked language model



3. Objectives, in one table

	GPT	BERT
Stack	decoder	encoder
Mask	causal	none on keys (random tokens hidden)
Loss	$p(x_t \mid x_{1:t-1})$	$p(x_{\text{masked}} \mid x_{\text{rest}})$
Default use	generate, chat, complete	classify, tag, embed a span
Peek at the future?	no	yes, by design

Modern “BERT-like” encoders (RoBERTa, embedding models) keep the bidirectional idea. Modern LLMs are GPT-style stacks, often with extra alignment (Weeks 8–9). Encoder–decoder models (T5, BART) exist;

they are the original transformer, not a third mystery architecture.

Do not fine-tune BERT as if it were GPT. A [MASK] model is not a chat decoder. You *can* classify with a decoder via a prompt; you cannot stream tokens from BERT without a separate head.

BERT-base, one line: $N = 12$, $d = 768$, 12 heads, about 110M parameters. GPT-1 was the same size on a decoder. The fork is the mask and the loss, not the width.

4. Scaling, briefly

The same block, bigger: more layers, wider d , more heads, more data. Week 7 is scaling laws and efficiency. This week: if the mask is causal, you have a language model; if not, you have a contextual encoder.

For the project: generation, RAG, and tool-use sit on a decoder. Classification into a small label set can still use an encoder, or a decoder with a prompt.

5. Teaching this note

~**16 minutes**. Static vs contextual *bank*, then the two attention triangles, then the objective table, then the 2×2 causal-softmax example. Play Karpathy **Let's build GPT 0:00–12:00** (tokens in, next token out). Then **0:00–8:00** of CodeEmporium BERT (MLM / [MASK]). If you only have one clip, keep Karpathy and assign BERT.

6. Worked example

Two tokens, raw scores $S = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$.

GPT (causal): illegal future is S_{12} . Set it to $-\infty$:

$$S_{\text{GPT}} = \begin{bmatrix} 1 & -\infty \\ 3 & 4 \end{bmatrix}.$$

Softmax row 1: $[1, 0]$. Row 2: $\text{softmax}([3, 4]) \approx [0.27, 0.73]$ because $e^3 \approx 20.1$, $e^4 \approx 54.6$.

BERT (bidirectional): softmax both rows of the original S . Row 1: $\text{softmax}([1, 2]) \approx [0.27, 0.73]$. Token 1 **does** look at token 2.

Objective contrast: GPT loss on this pair is $-\log p(x_2 | x_1)$. BERT, if token 1 is [MASK], is $-\log p(x_1 | x_2)$ (and any other masked sites).

7. Where students get stuck

- Using BERT when the demo must **stream** tokens.

- Masking GPT scores with 0 instead of $-\infty$.
 - Calling every transformer “a GPT.”
 - Thinking [MASK] is the same object as a causal key mask.
-

8. Video

Watch [Karpathy: Let's build GPT](#) through the first attention / causal-mask implementation (start **0:00–12:00** in class; the attention code is later if you assign homework).

Also [CodeEmporium: BERT Neural Network — EXPLAINED!](#). Pause on masked language modeling and next-sentence prediction. You do not need to train BERT today.

9. Practice

1. Why is BERT the wrong pretrained model if your demo must stream tokens?
2. A sentiment classifier: encoder or decoder? Give one reason for each.
3. What does a causal mask set the illegal scores to before the softmax, and why not zero?
4. Scores $\begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$, causal mask. Write A . (Uniform among **legal** keys.)
5. BERT hides 15% of tokens. In a sequence of length 20, how many tokens are masked in expectation? If the loss is only on those positions, how many terms enter the mean that step?
6. *bank* in “river bank” versus “money bank”: what did Week 1’s table do, and what does BERT do?
7. Name one encoder-only, one decoder-only, and one encoder–decoder model. Which one is the default stack for this course’s generation projects?